EMBEDDINGS_PROVIDER

HelixLLM CPU-Capable Embeddings Provider — Design Spike (P3-T6 / CPU tier)

Status	DESIGN (spike before implementation, §11.4.6 — do not code the provider until this is agreed)
Scope	The **CPU-only** embeddings capability — NO GPU, NO VRAM-broker dependency, so it ships **before** the P0/P1 GPU chain
Owns	The CPU slice of implementation-plan item **P3-T6** (`04_implementation_plan.md` line 84)
Created	2026-07-06 · Revision 1 · Track (`T1/main`) · Branch `feature/helixllm-full-extension`
Grounding	`submodules/helix_llm/docs/API_CONTRACT.md` §4.4 · `submodules/helix_llm/docs/VRAM_BROKER.md` §2 (CPU-only tier) · `docs/research/07.2026/00_master/04_implementation_plan.md` P3-T6 · `docs/research/07.2026/04_embeddings_rag/04_embeddings_rag.md`

> **Anti-bluff (§11.4.6):** every RAM/latency figure in this document is an **estimate to be measured** (`nvidia-smi` is irrelevant here — this is a CPU-resident service; the budget MUST be replaced by on-host `RSS` + wall-clock latency deltas once the service runs). No benchmark below is a captured measurement; each is flagged `(EST — measure)`.

0. Why this can ship before the GPU tiers

The programme's critical path (`04_implementation_plan.md` §"Critical-path sequencing") hard-serialises **P0 host GPU foundation → P1 serving core + VRAM broker → P2 gateway** before the P3 extended capabilities. That gating exists because the coder fleet, VLM, image/video generators, and the GPU embeddings tier all contend for the single **RTX 5090 · 32 GB** (`VRAM_BROKER.md` §1).

The **CPU embeddings provider designed here takes no GPU reservation** — it is a member of the broker's **CPU-only tier** (`VRAM_BROKER.md` §2: *"CPU-only … no GPU reservation … 0 GB GPU"*; the broker's admission `Class "embed"` (`VRAM_BROKER.md` §4) resolves to a **zero-byte VRAM lease** for this variant). It therefore does **not** depend on P0-T1 GPU passthrough, the P0-T3 sm_120 builds, or the P1-T4 residency broker being implemented. It only depends on the gateway route that **already exists** in the shipped HelixLLM binary (`API_CONTRACT.md` §4.4 — `POST /v1/embeddings` is registered TODAY at `internal/gateway/router.go:76`). This makes it the **earliest-shippable P3 capability** and a clean vertical slice to validate the P3 per-capability template (container → gateway route → verifier probe → HelixQA bank) with zero GPU risk.

> **Honest boundary vs the source docs (§11.4.6).** `VRAM_BROKER.md` §2 lists the CPU-only tier members as *"Qdrant, HelixMemory, NLLB (CTranslate2 int8 … can also be CPU), Tesseract"* — it does **not** name "embeddings" in that row verbatim. What the broker *does* provide is an `embed` **admission class** (`VRAM_BROKER.md` §4) whose GPU cost, for this CPU variant, is zero. This document **honours the task's directive to treat embeddings as CPU-capable** by defining the CPU tier explicitly and reconciling the broker table: the GPU P3-T6 model (`Qwen3-Embedding-4B`, `04_implementation_plan.md:84`) remains a **warm-swappable GPU** member for later; the CPU model defined here is the **ship-first** member with a 0-VRAM broker class. This is a documentation reconciliation, not a contradiction — flagged so a reviewer can confirm.

---

1. Engine choice — evidence-based comparison

1.1 The candidates

#	Engine	Serves an OpenAI `/v1/embeddings` route natively?	CPU-first?	Reranker in same toolkit?	New engine to the tree?
A	**HF Text Embeddings Inference (TEI)** — CPU image	**Yes** (`/v1/embeddings` under the `/v1` prefix)	**Yes** (dedicated `cpu-1.9` + `cpu-arm64-1.9` images)	**Yes** (`/rerank`, XLM-R/GTE/ModernBERT sequence-classification)	Container only (no Go dep)
B	**llama.cpp `llama-server --embeddings`**	**Yes** (`--embeddings` flag enables the OAI-compatible `/v1/embeddings`)	Yes (CPU build)	No (separate rerank path only)	**Already in-tree** (`dependencies/LLama_CPP`, router image)
C	CTranslate2 / sentence-transformers (Python)	**No** — a library, needs a custom FastAPI OpenAI shim	Yes	Partial (sbert CrossEncoder, still custom)	New Python service + custom route
D	Dedicated ONNX runtime (FastEmbed / Optimum-Intel)	**No** — client libraries, need a custom server shim	Yes (ONNX INT8 on CPU)	FastEmbed rerank exists, still a lib	New Python service + custom route

1.2 Decision

Primary engine: HF Text Embeddings Inference (TEI), CPU image `ghcr.io/huggingface/text-embeddings-inference:cpu-1.9` (x86_64) / `cpu-arm64-1.9` (aarch64). **Documented fallback lane: llama.cpp `llama-server --`**

`embeddings` (option B), because it is already incorporated in the tree and adds no new engine.

1.3 Justification (cited)

TEI is the exact engine P3-T6 already commits to — `04_implementation_plan.md:84` reads *"Qwen3-Embedding-4B (+BGE-M3 sparse) + bge-reranker-v2-m3 **via TEI**"*. Choosing TEI for the CPU tier is therefore **reuse, not a new decision** (§11.4.74 extend-don't-reimplement): the CPU variant simply pins the CPU image and a smaller model, and the later GPU variant swaps to a GPU image + a bigger model behind the **same** engine and the **same** gateway route.

1. **Native OpenAI `/v1/embeddings` on CPU, no custom glue.** TEI ships dedicated CPU containers and exposes the OpenAI-compatible route under `/v1` — `curl http://localhost:8080/v1/embeddings` returns the OpenAI shape; CPU is a first-class target, and **ARM64 is supported CPU-only**. (Options C and D would each require us to hand-write an OpenAI route — new code = new §11.4/§107 bluff surface.) Sources: TEI — Supported models & hardware (accessed 2026-07-06) lists the `cpu-1.9`/`cpu-arm64-1.9` images and *"can be used on CPU … ARM64 (aarch64) is supported for both CPU-only and CUDA"*; TEI — Quick Tour and TEI README (accessed 2026-07-06) document `text-embeddings-router --model-id … --port 8080` and the `/v1/embeddings` OpenAI route.
2. **The reranker P3-T6 also needs comes free in the same toolkit.** TEI's supported-rerankers table includes `BAAI/bge-reranker-base`/`-large` and `Alibaba-NLP/gte-reranker-modernbert-base` — one engine covers both the `embed` and `rerank` halves of P3-T6, on CPU. Source: TEI — Supported models (accessed 2026-07-06), "Supported re-rankers and sequence classification models".
3. **Token-based dynamic batching is the CPU throughput lever.** TEI batches concurrent requests by token budget, which is what makes a CPU embedder useful for bulk RAG ingestion rather than one-doc-at-a-time. Options C/D would need us to build batching ourselves. Source: TEI README (accessed 2026-07-06), "token-based dynamic batching".
4. **The ONNX-CPU advantage of option D is already inside TEI.** FastEmbed / Optimum-Intel win on CPU by running INT8 ONNX kernels; TEI's CPU backend already uses optimized CPU inference (candle/ONNX-class kernels, Intel container variant available), so we get the ONNX-CPU speedup **behind a real OpenAI server** instead of a bespoke shim. Sources: Qdrant FastEmbed and CPU-Optimized Embeddings with Optimum-Intel (accessed 2026-07-06) establish the ONNX-INT8-on-CPU technique; TEI — Intel container (accessed 2026-07-06) shows TEI captures it.

Why not the fallback as primary: llama.cpp `--embeddings` (option B) is a genuinely strong, zero-new-engine choice — the OAI-compatible `/v1/embeddings` route is real (`llama-server … --embeddings` enables it) and GGUF quantisation is excellent on CPU. Sources: llama.cpp server README and llama.cpp embeddings tutorial (Discussion #7712) (accessed 2026-07-06). It is kept as the **fallback lane** rather than primary because: (a) it does **not** cover the reranker half of P3-T6 through the same endpoint; (b) embedding-model coverage is limited to architectures with GGUF + llama.cpp support (nomic-bert, bge/e5 via the bert path) versus TEI's broader current list; (c) pooling must be set by hand (`--pooling cls|mean`). Selecting TEI primary + llama.cpp fallback gives one clean primary and a no-new-dependency degraded path.

1.4 CPU model selection (ship-first) — measured, not asserted

The engine is fixed; the **model is a config value** injected per §CONST-046 / §11.4.35, never hardcoded. Candidate CPU models, all TEI-supported and CPU-runnable, ranked by the quality/size trade the CPU tier cares about:

Model	Params	Dim	Ctx	License	Role
`nomic-ai/nomic-embed-text-v1.5`	137M	768 (Matryoshka → 512/256)	8192	Apache-2.0	**Default CPU model** — best quality/size + long context
`BAAI/bge-small-en-v1.5`	33M	384	512	MIT	Ultra-light lane (lowest RAM/latency)
`Qwen/Qwen3-Embedding-0.6B`	509M	up to 1024	long	Apache-2.0	Higher-quality CPU lane (heavier); shares model family with the GPU P3-T6 `Qwen3-Embedding-4B`

Source for model IDs, sizes, MTEB ranks and CPU-viability: TEI — Supported models (accessed 2026-07-06); Nomic long-context + Matryoshka corroborated by nomic-ai/nomic-embed-text-v1.5 (accessed 2026-07-06).

Recommendation: default `nomic-ai/nomic-embed-text-v1.5` — Apache-2.0, 137M, 768-dim, 8192-token context (long-doc RAG on CPU), Matryoshka truncation lets us trade dim for speed without a model swap. `bge-small` is the "fits anywhere" fallback; `Qwen3-Embedding-0.6B` is the "same family as the GPU tier"

upgrade path so the CPU→GPU migration is a model-size change, not an ecosystem change.

1.5 CPU RAM / latency budget — ESTIMATES to be measured (§11.4.6)

All figures (`EST — measure`); replace with on-host `RSS` + p50/p95/p99 wall-clock captured under `docs/qa/<run-id>/embeddings/` before any PASS:

Model	Weights (fp32) (EST)	Weights (INT8) (EST)	Resident RSS incl. runtime (EST)	Single short-doc latency (EST)	Batched throughput (EST)
bge-small (33M)	~130 MB	~35 MB	~0.3–0.6 GB	~5–20 ms	high
nomic-v1.5 (137M)	~550 MB	~140 MB	~0.7–1.2 GB	~10–40 ms	med-high
Qwen3-Emb-0.6B (509M)	~2.0 GB	~0.5 GB	~2.5–3.5 GB	~30–120 ms	med

Host has **64 cores** / **251 GiB** (`RESUME.md` live-state), so even the largest CPU model is comfortably resident with room for TEI's batch buffers; the CPU tier never competes with the GPU coder fleet for the 32 GB card. Thread count is a tuning knob (`--tokenization-workers` / OMP threads) to bound the observer-effect on the shared host (§11.4.128-adjacent hygiene). **These numbers gate nothing until measured.**

---

2. API contract — OpenAI-compatible `/v1/embeddings` (consistent with API_CONTRACT.md §4.4)

The HelixLLM binary **already registers** `POST /v1/embeddings` (`API_CONTRACT.md` §2 table → `internal/gateway/router.go:76`, `HandleEmbeddings`, **API-key** authed). This provider becomes the **real backend** for that route — replacing the current behaviour, which delegates to a knowledge-layer `Embedder` when present and otherwise **returns a zero vector of dim 1536** (`API_CONTRACT.md` §4.4, `internal/gateway/openai.go:641-661`). That zero-vector stub is exactly the bluff the acceptance test in §4 must kill.

2.1 Request — MUST match `api.EmbeddingRequest` (`pkg/api/openai.go:142-145`)

Today's struct is `{model, input(string|[]string)}` (`API_CONTRACT.md` §4.4). The provider keeps that shape and MAY extend it with OpenAI-superset optional fields; extending the struct is a **flagged, reviewed change** to `pkg/api/openai.go`, not an ambient assumption (§11.4.6):

```
POST /v1/embeddings
Authorization: Bearer <key>            # API-key middleware, router.go:63-
Content-Type: application/json
{
  "model": "helix-embed",              # Helix alias → backing TEI model (s
  "input": ["The cat sat on the mat.", "A feline rested on the rug."],
  "encoding_format": "float",          # OPTIONAL superset — "float" (defau
  "dimensions": 512                    # OPTIONAL superset — Matryoshka tru
}
```

- `model` — a **Helix alias**, not a raw HF id (CONST-036/037: models come from the provider layer). Empty `model` currently defaults to `text-embedding-ada-002` (`API_CONTRACT.md` §4.4, `openai.go:598-601`); the provider maps that legacy default → the configured CPU model so old clients keep working.
- `input` — `string` or `[]string` (unchanged; matches the struct).
- `encoding_format`, `dimensions` — **additive** OpenAI-superset fields; omitted-field behaviour is identical to today (float, native dim). Only added if `api.EmbeddingRequest` is extended in a reviewed diff; until then they are accepted-and-ignored, documented as `UNCONFIRMED` per §11.4.6.

2.2 Response — MUST match `api.EmbeddingResponse` (`pkg/api/openai.go:147-158`)

Byte-shape identical to `API_CONTRACT.md` §4.4 — `{object:"list", data:[{object:"embedding", embedding:[]float64, index}], model, usage}`:

```
{
  "object": "list",
  "data": [
    {"object": "embedding", "index": 0, "embedding": [0.0123,
        -0.0456, "…768 floats…"]},
    {"object": "embedding", "index": 1, "embedding": [0.0119,
        -0.0448, "…768 floats…"]}
  ],
  "model": "helix-embed",
```

```
  "usage": {"prompt_tokens": 14, "total_tokens": 14}
}
```

The gateway's `HandleEmbeddings` translates the OpenAI response TEI emits into this exact struct (or passes it through, since TEI is already OpenAI-shaped). `GET /v1/models` MUST advertise `helix-embed` with `owned_by: "helix"` (`API_CONTRACT.md` §4.3) so LLMsVerifier and clients discover it (CONST-036).

2.3 Model aliasing + provider registration

- A config-driven `ProviderDescriptor` (mirrors P2-T4, `04_implementation_plan.md:73`) registers the CPU embeddings endpoint with LLMsVerifier so the `helix-embed` alias, its dimension, and its capability flag are **verifier-sourced, not hardcoded** (CONST-036/037/040).
- Alias → backing-model map lives in HelixLLM config (env / YAML), never a source literal (§CONST-046). Adding a model = a config edit, no code change.

2.4 Error shape

Unchanged from the gateway: OpenAI-error JSON `{"error":{"message":…,"type":"invalid_request_error"}}` (`API_CONTRACT.md` §3, `auth.go:58-64`) for auth/validation; a `503` with the same envelope when the backing container is not yet warm (honest "warming", never a silent zero-vector — see §4).

---

3. Containerization — rootless podman via the `containers` submodule, NO GPU

Per §11.4.76 (containers-submodule mandate) + §11.4.161 (rootless runtime) the service is booted **through** `vasic-digital/containers` (`pkg/boot` / `pkg/compose` / `pkg/health`), never a hand-run `podman`/`docker` command, and never rootful.

3.1 Image + run shape (illustrative — config-injected, no hardcoded host §CONST-045)

- **Image:** `ghcr.io/huggingface/text-embeddings-inference:cpu-1.9` (x86_64) or `:cpu-arm64-1.9` (aarch64), selected by `uname -m` per §11.4.81 cross-platform parity. Pinned by digest in production (§11.4.76 clause 2).
- **NO GPU:** the run spec contains **no** `--device nvidia.com/gpu=all` and **no** `--security-opt=label=disable` GPU flag (contrast the P0 GPU proof in `04_implementation_plan.md:44`). This is the structural guarantee it needs no P0.

- **Model source:** `--model-id <hf-id>` with a persistent HF-cache volume (`-v helixllm-tei-cache:/data`), OR a pre-fetched local model mounted read-only (`-v $MODELS_DIR:/data:ro`) where `$MODELS_DIR` is **injected** (env), never a literal. The model weights are a §11.4.77 re-obtain artefact (gitignored; `fetch_models.sh`-class script downloads from HF — matches `04_implementation_plan.md` P7-T1 `fetch_models.sh`).
- **Port:** a config-injected host port (e.g. `:18435`, distinct from the coder fleet's `:18434` in `RESUME.md`), reached by the HelixLLM gateway; `--network` per the containers-submodule compose spec, not a hand-set flag.
- **Boot is part of the test entry point** (§11.4.76 on-demand-infra invariant): the HelixQA embeddings bank boots the container via the submodule, waits on `pkg/health` (TEI `/health`), then drives the gateway route — a short-circuit fake that skips the boot is a §11.4 violation.
- **Broker interaction:** the VRAM broker's `Acquire(ctx, "embed")` (`VRAM_BROKER.md` §4) returns a **0-byte lease** for this CPU variant — the broker records the service as CPU-only tier and takes no GPU reservation, so the service is admissible even with the whole card committed to the coder fleet. No broker code is required to ship the CPU provider (the broker is a P1 component; the 0-VRAM class is just its CPU-tier default).
- **Catalogue-Check (§11.4.74):** `extend vasic-digital/containers@<sha>` — add a `tei-embeddings` compose profile to the containers submodule if one does not exist; never an in-project ad-hoc compose file.

3.2 Cross-platform + resource hygiene

- ARM64 host → `cpu-arm64-1.9` image (TEI documents aarch64 CPU support), chosen by runtime `uname` dispatch (§11.4.81).
- Bounded CPU threads + memory limit on the container (§12.3 container hygiene) so the embedder never starves the developer host (§12.6) — the limits are config-injected and captured as evidence during the acceptance run.

---

4. Anti-bluff acceptance — the ONE machine-checkable runtime signature (§11.4.108)

Definition of done for this provider: on a **clean deploy** (§11.4.108/§11.4.139 — container freshly booted via the containers submodule, gateway pointed at it), the following single machine-checkable signature verifies and is captured to `docs/qa/<run-id>/embeddings/`:

> **RUNTIME SIGNATURE (embeddings semantic-order):** POST a **sentence triple** to `POST /v1/embeddings` — `A` = *"The cat sat on the mat."*, `A'` = *"A feline rested on the rug."* (paraphrase of A), `U` = *"Quarterly*

revenue rose four percent." (unrelated). Compute cosine similarity on the returned vectors and assert **`cos(A, A') − cos(A, U) ≥ margin`** (margin calibrated on the project's own fixtures per §11.4.107(13), e.g. ≥ 0.15), AND every returned vector has the **expected dimension** and a **non-zero L2 norm**, AND the vectors are **deterministic** across two identical requests (§11.4.50). The captured artefact is the raw `/v1/embeddings` JSON + the computed cos-similarity matrix + PASS/FAIL verdict with its evidence path.

This is a genuine end-to-end proof: it can only PASS if a real model produced meaning-bearing vectors through the real gateway route — it is impossible to satisfy with the current **dim-1536 zero-vector stub** (`openai.go:641-661`), which fails both the non-zero-norm check and the semantic-order margin.

4.1 Golden-good / golden-bad self-validation (§11.4.107(10))

The cosine-order **analyzer itself is mutation-proofed** with a fixture pair, wired into the meta-test:

- **golden-good fixture** — a captured real `/v1/embeddings` response for the `A/A'/U` triple where the margin genuinely holds → the analyzer MUST return **PASS**.
- **golden-bad fixtures** (each MUST return **FAIL**, proving the analyzer cannot be fooled):
  1. **zero-vector** / **constant-vector** response (the exact §4.4 stub shape) → fails non-zero-norm + margin.
  2. **shuffled-order** response where `cos(A,U) > cos(A,A')` → fails the semantic-order margin (catches a model that emits vectors but no meaning / a wrong-model mis-wire).
  3. **wrong-dimension** response → fails the dimension check.

Paired §1.1 mutation: strip the margin/non-zero-norm/determinism assertion from the analyzer → the zero-vector golden-bad fixture PASSes → the gate FAILs. That mutation is the mechanical proof the acceptance test is not itself a bluff.

4.2 Higher-order + resilience proofs (compose, do not replace §4)

- **Retrieval Recall@K on a fixture** (the P3-T6 signature, `04_implementation_plan.md:84`): embed a small labelled corpus + queries, assert Recall@K above a fixture-calibrated floor — the "does it actually power RAG" proof beyond pairwise cosine.
- **Determinism** / **re-runnability** (§11.4.50/§11.4.98): identical input → byte-identical vectors; the whole bank PASSes at `-count=3`.
- **Stress** + **chaos** (§11.4.85): batch of N≥100 inputs (throughput + p50/p95/p99 captured), N≥10 concurrent callers (no deadlock/leak), boundary inputs

(empty string, max-context-length doc, unicode) each categorised; chaos = container SIGKILL mid-request → gateway returns an honest `503 warming`, never a silent zero-vector.
- **Feature-class evidence (§11.4.69):** the closed sink-side taxonomy has no `embedding` class today; this provider **adds one** (taxonomy is *open to additions, never contraction* per §11.4.69) — evidence shape = the captured cosine-order + non-zero-norm artefact above. Flagged for the §11.4.69 taxonomy owner.

4.3 Four-layer verification (§11.4.108)

1. **SOURCE** — provider + gateway wiring committed; pre-build grep gate.
2. **ARTIFACT** — the TEI CPU image pulled + pinned by digest; `pkg/health` green.
3. **RUNTIME-ON-CLEAN-TARGET** — the §4 runtime signature verifies against a freshly-booted container (not a stale one) — the definition of done.
4. **USER-VISIBLE** — an OpenAI-compatible client (HelixCode / any CLI agent) points at `/v1/embeddings`, gets real vectors, and a downstream RAG query returns the right chunk (the P3-T7 consumer).

---

5. Open questions (resolve before coding)

- **Q1** Extend `api.EmbeddingRequest` with `encoding_format` + `dimensions`, or keep the strict `{model,input}` struct and handle Matryoshka server-side only? (Leaning: additive fields, reviewed diff — many OpenAI clients send them.)
- **Q2** Default CPU model — `nomic-embed-text-v1.5` (recommended) vs `Qwen3-Embedding-0.6B` (same family as the GPU tier). Decide on measured Recall@K + latency, not the estimates in §1.5.
- **Q3** Does the `containers` submodule already have a TEI/embeddings compose profile, or is this a §11.4.74 `extend` PR? (Investigate before scaffolding.)
- **Q4** Reranker (`bge-reranker`, the other half of P3-T6) — same TEI CPU container with a `/rerank` route, or a second container? (TEI serves one model per process; likely a second CPU container in the same compose profile.)

---

6. Composition footer — constitutional anchors touched

- **§11.4.6** (no-guessing) — every RAM/latency figure flagged `(EST — measure)`; the VRAM_BROKER CPU-tier reconciliation flagged, not asserted.
- **§11.4.74** (extend-don't-reimplement) — reuse TEI (the engine P3-T6 already names) + extend the containers submodule; no bespoke embedding server.
- **§11.4.76** / **§11.4.161** (containers submodule / rootless) — boot via `pkg/boot`+`pkg/compose`+`pkg/health`, rootless podman, no GPU device.
- **§11.4.77** (re-obtain mechanism) — model weights gitignored + `fetch_models.sh`.
- **§11.4.81** (cross-platform parity) — `cpu-1.9` (x86_64) vs `cpu-arm64-1.9` (aarch64) chosen by runtime dispatch.
- **§11.4.99** / **§11.4.150** (latest-source + deep multi-angle research) — engine decision cited to LATEST upstream docs, ≥2 distinct angles (TEI, llama.cpp, FastEmbed/Optimum, sentence-transformers).
- **§11.4.107(10)/(13)** (self-validated analyzer + fixture-calibrated thresholds) — golden-good/golden-bad cosine-order analyzer, project-calibrated margin.
- **§11.4.108** / **§11.4.139** (four-layer runtime-signature on a clean target) — the §4 acceptance signature is the definition of done.
- **§11.4.69** (sink-side evidence taxonomy) — adds an `embedding` feature class.
- **§11.4.85** / **§11.4.98** / **§11.4.50** (stress+chaos / full-automation / determinism).
- **§11.4.135** (standing regression guard) — the §4 signature registers as a permanent guard.
- **CONST-036/037/040** (LLMsVerifier single source of truth; capability flags verifier-sourced) — `helix-embed` alias + dimension + capability from the verifier, never hardcoded.
- **CONST-046** (no hardcoded content) — model ids / host / port config-injected.

Sources verified

Deep-research 2026-07-06: - https://huggingface.co/docs/text-embeddings-inference/en/supported_models - https://huggingface.co/docs/text-embeddings-inference/en/quick_tour - https://github.com/huggingface/text-embeddings-inference - https://huggingface.co/docs/text-embeddings-inference/en/intel_container - https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md - https://github.com/ggml-org/llama.cpp/discussions/7712 - https://huggingface.co/nomic-ai/nomic-embed-text-v1.5 - https://github.com/qdrant/fastembed - https://huggingface.co/blog/intel-fast-embedding

(Negative finding, §11.4.99(B): the canonical OpenAI `/v1/embeddings` reference page `platform.openai.com/docs/api-reference/embeddings/create` returned HTTP 403 to automated fetch on 2026-07-06; the request/response shape

in §2 is instead grounded in the in-tree `API_CONTRACT.md` §4.4 source-verified struct citations + the TEI OpenAI-compatible route docs above, which is the authoritative shape for THIS system.)